TP 1 : Manipulation et Visualisation de Signaux de Parole

Objectif : Maîtriser l'importation, l'écoute, la visualisation et la modification des propriétés temporelles d'un signal audio .wav.

---

1. Chargement et Écoute Interactive

Avant de traiter un signal, il faut savoir l'extraire du disque et vérifier son contenu à l'oreille.

Travail à faire

1. Chargez un fichier .wav de votre dataset. Quelles sont les deux variables récupérées ?
2. Affichez la fréquence d'échantillonnage (*rate*) et calculez la durée totale du signal en secondes.
3. **Écoute :** Utilisez la fonction dédiée pour écouter le signal directement dans votre interface. Testez l'option *autoplay=True*.

Lecture : scipy.io.wavfile.read(filename)

- **Entrée :** Chemin du fichier (str).
- **Sortie :** Un tuple (rate, data). rate est un entier, data est un tableau Numpy contenant les amplitudes.

Écoute : IPython.display.Audio(data, rate=*rate*)

- **Entrée :** Le tableau de données et la fréquence d'échantillonnage.
- **Sortie :** Un widget de lecture audio.

---

2. Visualisation : Création d'une fonction générique

La visualisation est cruciale en TAP pour identifier les zones de silence, les occlusives ou les voyelles.

Travail à faire

Réalisez une fonction Python nommée *visualiser_signal(sgn, fs=8000, en_secondes=False)* qui :

1. Affiche le signal avec un titre, et des labels explicites pour les axes (*x* et *y*).
2. Si en_secondes est True, l'axe des abscisses doit afficher le temps en secondes (utilisez *rate* pour convertir les indices). Sinon, la fonction affiche le numéro de l'échantillon.

Graphisme : matplotlib.pyplot

- plt.plot(données) : Trace la courbe.
- plt.xlabel("Nom") / plt.ylabel("Nom") : Nomme les axes.
- plt.title("Titre") : Ajoute un titre.
- **Astuce pour le temps :** Le vecteur temps se crée avec *np.arange(len(sgn)) / rate*.

---

3. Analyse locale : Le Zoom

Pour observer la périodicité d'un signal (ex: une voyelle voisée), il faut zoomer sur une fenêtre très courte (quelques millisecondes).

Travail à faire

Visualisez un segment précis du signal (par exemple entre l'échantillon 1800 et 2200) en utilisant deux méthodes différentes :

1. **Méthode par isolation :** Créez un nouveau tableau contenant uniquement le segment (slicing) et tracez-le.
2. **Méthode par fenêtre d'affichage :** Tracez le signal complet, mais limitez l'affichage de l'axe X.
3. Localisez les zones périodiques du signal et déduisez la fréquence de ces zones.

Pour limiter l'axe des x : plt.xlim(debut, fin)

- **Entrée :** Les bornes min et max de l'index (ou du temps) à afficher à l'écran.

---

4. Normalisation de l'amplitude

Certains signaux sont enregistrés avec un volume trop faible. La normalisation standardise l'amplitude.

Travail à faire

1. Calculez la moyenne (μ) et l'écart-type (σ) de votre signal avec numpy (mean, std).
2. Appliquez la formule : $sgn_{normalisé} = \frac{sgn - \mu}{\sigma}$
3. Tracez le signal normalisé. Que remarquez-vous sur l'échelle de l'axe *y* ?

---

5. Rééchantillonnage et impact sonore

Le rééchantillonnage change la "résolution" temporelle du son.

Travail à faire

1. Ecoutez le signal avec une fréquence d'échantillonnage (rate) doublée puis divisée par 2 : (ex : 8000 -> 16000 puis 8000 -> 4000), que remarquez-vous ?
2. Changez la fréquence d'échantillonnage du signal à 2000Hz en utilisant la fonction *resample* puis écoutez le résultat et comparez avec le signal original.

Documentation Technique

Transformation : scipy.signal.resample(sgn, num)

- **Entrée :** sgn (le signal original), num (le nombre d'échantillons attendu en sortie).
- **Sortie :** Le nouveau signal modifié.

La fonction resample ne demande pas la nouvelle fréquence, mais le nombre total d'échantillons final.

- Si vous avez 16 000 points à 16 kHz (soit 1 seconde de son).
- Pour passer à 8 kHz, il ne vous faut plus que 8 000 points pour la même seconde.
- **Formule :** $num = len(original\ signal) \times \frac{New\ rate}{Old\ rate}$